

FIRAT ÜNİVERSİTESİ
Mühendislik Fakültesi
Bilgisayar Mühendisliği Bölümü

BMÜ326 YAZILIM MÜHENDİSLİĞİ
DÖNEM PROJESİ RAPORU

[PROJE ADI]

Grup No	[Grup Numarası]
Takım Üyesi 1	[Öğrenci No] – [Ad Soyad]
Takım Üyesi 2	[Öğrenci No] – [Ad Soyad]
Takım Üyesi 3	[Öğrenci No] – [Ad Soyad]
Ders Sorumlusu	[Öğretim Üyesinin Adı]
GitHub Repo Linki	[https://github.com/.../...]
Jira Proje Linki	[https://....atlassian.net/...]
Teslim Tarihi	[GG/AA/YYYY]

İÇİNDEKİLER

[İçindekiler tablosu teslim öncesi Word üzerinden "Başvurular > İçindekiler Tablosu" menüsünden güncellenerek buraya eklenecektir.]

1. GİRİŞ VE PROJE TANIMI

1.1 Projenin Amacı ve Kapsamı

[Projenin amacını ve kapsamını açıklayınız. Hangi probleme çözüm getirildiğini, sistemin temel işlevlerini ve hedef kullanıcı kitlesini belirtiniz. Projenin sınırlarını, neleri kapsadığını ve neleri kapsamadığını açıkça yazınız.]

1.2 Hedef Kullanıcılar

[Sistemde rol alacak farklı kullanıcı türlerini tanımlayınız. Her kullanıcı tipi için kısa bir profil (rol, ihtiyaç, beklenti) belirtiniz. Örneğin bir e-ticaret sistemi için: müşteri, satıcı, sistem yöneticisi, kargo görevlisi vb.]

1.3 Kullanılan Teknolojiler ve Araçlar

[Aşağıdaki tabloya proje kapsamında kullanılan teknolojileri, kütüphaneleri ve araçları yazınız.]

Kategori	Teknoloji / Araç	Açıklama / Kullanım Amacı
Programlama Dili	[Örn. C# / Java / Python / JS]	[Hangi katmanda kullanıldığı]
Backend Çatısı	[Örn. ASP.NET Core / Spring Boot / Django / Express.js]	[REST API / iş mantığı]
Frontend Çatısı	[Örn. React / Angular / Vue / Blazor]	[Kullanıcı arayüzü]
ORM Aracı	[Örn. Entity Framework Core / Hibernate / SQLAlchemy / Sequelize / Prisma]	[Veritabanı erişim katmanı]
Veritabanı	[Örn. PostgreSQL / MySQL / SQL Server / MongoDB]	[Kalıcı veri saklama]
Birim Test Çatısı	[Örn. xUnit / JUnit / pytest / Jest]	[Unit test geliştirme]
Proje Yönetimi	Jira (Kanban)	İş takibi, çevik süreç yönetimi
CI/CD (Bonus)	[GitHub Actions]	[Otomatik build, test, deploy]
Bulut Ortamı (Bonus)	[Render / Railway / Supabase / Neon / MongoDB Atlas]	[Production deployment / bulut DB]
Yapay Zekâ API (Bonus)	[OpenAI / Gemini / Claude / Hugging Face]	[Sistem içinde AI kullanım senaryosu]

2. PROJE GEREKSİNİMLERİ

Bu bölümde projenin kullanıcı hikâyeleri, kabul kriterleri, fonksiyonel ve fonksiyonel olmayan gereksinimleri ile iş kuralları sunulacaktır. Gereksinimler iteratif (artımsal) geliştirme süreci boyunca gözden geçirilecek ve gerektiğinde güncellenecektir.

2.1 Kullanıcı Hikâyeleri (User Stories) ve Kabul Kriterleri (Acceptance Criteria)

[Kullanıcı hikâyeleri "Bir <kullanıcı rolü> olarak, <istediğim özellik>'i, <amaç> için yapabilmek isterim." şablonuna göre yazılacaktır. Her hikâye için kabul kriterleri (Acceptance Criteria) maddeler hâlinde verilecektir.]

ID	Hikâye Başlığı	Kullanıcı Hikâyesi	Kabul Kriterleri
US-01	Üye Kaydı	Bir ziyaretçi olarak, sisteme üye olabilmek için e-posta ve şifreyle kayıt olabilmek isterim.	- E-posta benzersiz olmalıdır. - Şifre en az 8 karakter olmalıdır. - Başarılı kayıtta doğrulama e-postası gönderilmelidir.
US-02	[...]	[...]	[...]

2.2 Fonksiyonel Gereksinimler

[Aşağıda verilen örnek tabloya göre projenin fonksiyonel gereksinimlerini oluşturunuz. Her gereksinim ilgili bir kullanıcı hikâyesi ile ilişkilendirilebilir.]

ID	Gereksinim Adı	Açıklama	İlgili User Story
FR-01	Üye Kaydı	Sistem, kullanıcıların e-posta ve şifre ile yeni hesap oluşturmaya izin vermelidir.	US-01
FR-02	[...]	[...]	[...]

2.3 Fonksiyonel Olmayan Gereksinimler

[Performans, güvenlik, kullanılabilirlik, sürdürülebilirlik vb. nitelik gereksinimlerini belirtiniz.]

ID	Gereksinim Adı	Açıklama
NFR-01	Güvenlik	Şifreler hash algoritması (örn. bcrypt) ile saklanmalı, hassas bilgiler GitHub Secrets / .env üzerinden yönetilmelidir.
NFR-02	Performans	Listeleme sayfaları normal yük altında 2 saniye içinde yanıt vermelidir.
NFR-03	Sürdürülebilirlik	Veri erişim kodu, iş mantığından ORM ve Repository Pattern aracılığıyla ayrılmalıdır.

2.4 İş Kuralları ve Kısıtlamalar

[Proje ile ilgili iş mantığı kuralları ve kısıtlamalar belirtilecektir. Örneğin: • Aynı kullanıcı aynı ürünü ikinci kez sepete eklediğinde adet bir artırılır. • Sipariş durumu yalnızca "Beklemede", "Hazırlanıyor",

"Kargoda", "Tamamlandı", "İptal" değerlerinden biri olabilir. • Stok 0 olan bir ürün için sipariş oluşturulamaz.]

3. ÇEVİK YAZILIM GELİŞTİRME SÜRECİ (JIRA – KANBAN)

Proje, çevik (Agile) yazılım geliştirme yaklaşımının Kanban yöntemine göre Jira proje yönetim aracı kullanılarak yürütülmüştür. Bu bölümde Kanban panosunun yapısı, iş kalemlerinin (epic / story / task) tanımlanması ve iterasyon süreci belgelenmektedir.

3.1 İş Kalemleri Hiyerarşisi (Epic / Story / Task)

[Projedeki temel epic'leri, bunlara bağlı story ve taskları aşağıdaki örnek tablodaki gibi hiyerarşik olarak belirtiniz. Her Story "Üst Öğe" sütununda bağlı olduğu Epic'in anahtarını, her Task ise bağlı olduğu Story'nin anahtarını içermelidir. Başlık sütunundaki girintiler hiyerarşi seviyesini görsel olarak gösterir.]

Tip	Anahtar	Üst Öğe	Başlık	Başlangıç	Bitiş	Talimat Veren	Gerçekleştirilen
Epic	PRJ-1	—	Üyelik & Kimlik Doğrulama	01.03.26	30.03.26	[Ad Soyad]	[Ad Soyad]
Story	PRJ-2	PRJ-1	Üye Kaydı	01.03.26	08.03.26	[Ad Soyad]	[Ad Soyad]
Task	PRJ-3	PRJ-2	UserRepository sınıfının yazılması	01.03.26	03.03.26	[Ad Soyad]	[Ad Soyad]
Task	PRJ-4	PRJ-2	POST /api/auth/register endpoint'i	04.03.26	06.03.26	[Ad Soyad]	[Ad Soyad]
Story	PRJ-5	PRJ-1	Kullanıcı Girişi	09.03.26	15.03.26	[Ad Soyad]	[Ad Soyad]
Task	PRJ-6	PRJ-5	JWT token üretimi	09.03.26	11.03.26	[Ad Soyad]	[Ad Soyad]
Epic	PRJ-7	—	Ürün Yönetimi	16.03.26	12.04.26	[Ad Soyad]	[Ad Soyad]
Story	PRJ-8	PRJ-7	Ürün Listeleme	16.03.26	22.03.26	[Ad Soyad]	[Ad Soyad]
Task	PRJ-9	PRJ-8	ProductRepository (filtreleme)	16.03.26	18.03.26	[Ad Soyad]	[Ad Soyad]
Task	PRJ-10	PRJ-8	Ürün listeleme UI	19.03.26	22.03.26	[Ad Soyad]	[Ad Soyad]
Story	PRJ-11	PRJ-7	Ürün Arama	23.03.26	29.03.26	[Ad Soyad]	[Ad Soyad]
Epic	PRJ-12	—	[Diğer Epic Başlığı]	[GG.AA.YY]	[GG.AA.YY]	[...]	

Not: Renk kodlaması — turuncu satırlar Epic, açık mavi satırlar Story, açık gri satırlar Task'tır. "Anahtar" sütunu Jira'da otomatik üretilen kart numarasıdır (örn. PRJ-1). "Üst Öğe" sütunu, kartın bağlı olduğu üst düzey kartın anahtarını gösterir; Epic'ler için bu alan "—" olarak bırakılır. Tarihler Jira → kart detay → Start date / Due date alanlarından alınır.

3.2 Jira Ekran Görüntüleri

[KANBAN PANOSUNUN GENEL GÖRÜNÜMÜ (To Do / In Progress / Code Review / Done sütunları görünecek şekilde)]

*[2-3 JİRA KARTININ DETAY EKRANI – DEVELOPMENT PANELİNDE BAĞLI BRANCH,
COMMIT VE PR BİLGİLERİ]*

*[AYNI BELİRLENEN KARTLARIN ACTIVITY / HISTORY SEKMESİ – KART DURUMUNUN
"TO DO → IN PROGRESS → DONE" GEÇİŞLERİ]*

4. SÜRÜM KONTROLÜ – GIT VE GITHUB

4.1 GitHub Repository Yapısı

[Repository klasör yapısını aşağıdaki tabloya yazınız. Örn: src/, docs/, tests/, scripts/, .github/]

Klasör / Dosya	Açıklama
/src	Uygulamanın kaynak kodu (backend ve/veya frontend modülleri).
/tests	Unit test ve entegrasyon test dosyaları.
/docs	Tasarım dokümanları, UML diyagramları, API dokümantasyonu vb.
/.github	CI/CD workflow dosyaları (varsa).
README.md	Proje tanıtım dosyası.
.gitignore	Sürüm kontrolünden hariç tutulacak dosyalar.

[GITHUB REPO ANA SAYFA EKRAN GÖRÜNTÜSÜ (KLASÖR YAPISI GÖRÜNECEK ŞEKİLDE)]

4.2 README Dosyası

[README dosyanızın içeriğini özetleyiniz. README; proje açıklaması, kurulum adımları, çalıştırma talimatları, kullanılan teknolojiler, ekran görüntüleri ve takım üyeleri bölümlerini içermelidir.]

[ÖRNEK COMMIT GEÇMİŞİ EKRAN GÖRÜNTÜSÜ]

4.3 GitHub Tarafı

[GITHUB → BRANCHES SEKMESİ – feature/PRJ-* DALLARININ LİSTESİ]

[GITHUB → PULL REQUESTS LİSTESİ – AÇIK VE KAPALI PR'LAR GÖRÜNECEK ŞEKİLDE]

4.4 Pull Request ve Code Review Süreci

Geliştirme tamamlanan her feature branch, main branch'e doğrudan merge edilmemiş; bunun yerine Pull Request açılmış ve en az bir takım üyesinin onayı (approval) alındıktan sonra merge işlemi gerçekleştirilmiştir. PR sürecinde inceleyen üyeler hata, öneri ve olumlu geri bildirimlerini yorum olarak iletmış; gerekli durumlarda "changes requested" verilmiştir.

[Aşağıdaki ekran görüntülerini ekleyiniz.]

[3-4 TANE ÖRNEK FEATURE BRANCH'İN PULL REQUEST VE CODE REVIEW SÜRECİ İLE İLGİLİ CODE REVIEW YORUMLARI VE "REQUEST CHANGES / APPROVE" EKRAN GÖRÜNTÜLERİ]

5. YAZILIM MİMARİSİ VE TASARIMI

5.1 Sistem Mimarisi (Architectural Pattern)

[Projede uygulanan mimari deseni belirtiniz. Örn. Layered (N-Tier) Architecture, MVC, MVVM, Clean Architecture, Microservices, Hexagonal Architecture vb. Seçim gerekçesini açıklayınız.]

5.1.1 Mimari Diyagram

[Sistemin yüksek seviyeli mimari diyagramını ekleyiniz. Katmanlar (Presentation – Business – Data Access – Database) ve aralarındaki bağımlılıklar görünmelidir.]

[MİMARİ DİYAGRAM GÖRSELİ]

5.1.2 Katmanların Sorumlulukları

[Her katmanın hangi sorumlulukları üstlendiğini ve hangi sınıf/modül gruplarını barındırdığını yazınız.]

5.2 Tasarım Desenleri (Design Patterns)

[Projede kullanılan tasarım desenlerinin hepsini ayrı alt başlıklarda açıklayınız. Örn. Repository, Singleton, Factory, Dependency Injection vb.]

5.2.1 Repository Pattern

[Repository Pattern'i nasıl uyguladığınızı, oluşturduğunuz sınıfları, Repository Pattern'in amacına uygun nasıl kullandığınızı detaylı açıklayınız. CRUD, filtreleme, listeleme ve arama operasyonlarının Repository sınıfları üzerinden nasıl yürütüldüğünü gösteren kod parçalarını ekleyiniz.]

5.2.2 [Diğer Tasarım Deseni Adı]

[Kullanılan ikinci tasarım deseninin amacını, hangi probleme çözüm getirdiğini ve nerede uygulandığını yazınız. İlgili sınıf/diyagram örneğini ekleyiniz.]

6. VERİTABANI TASARIMI VE ORM

Veritabanı işlemleri; kullanıcı hikâyeleri ve kabul kriterleri doğrultusunda iteratif olarak tasarlanması. tüm veritabanı etkileşimleri yalnızca ORM (Object-Relational Mapping) üzerinden yapılması; ham SQL komutları doğrudan kullanılmaması, şema değişiklikleri migration ile yönetilmesi, tablo ilişkilerini (1-N, N-N vb.) ORM üzerinde modellenmesi beklenmektedir.

6.1 Varlık-İlişki (E-R) Modeli

[E-R diyagramını ekleyiniz. Tüm varlıkları, niteliklerini, ilişkileri ve ilişki türlerini gösteriniz.]

[E-R DİYAGRAMI]

6.2 İlişkisel Şema

[E-R diyagramından elde edilen ilişkisel şemaları, birincil ve yabancı anahtarlarıyla birlikte yazınız.]

6.3 ORM Konfigürasyonu ve Entity Sınıfları

[Kullanılan ORM aracını (örn. Entity Framework Core, Hibernate, SQLAlchemy, Sequelize, Prisma) ve DbContext / Session / Schema yapılandırmasını açıklayınız. En az bir entity sınıfının kodunu ekleyiniz.]

6.4 Tablo İlişkilerinin Modellenmesi

[Bire-çok (1-N) ve çok-çok (N-N) ilişkilerin ORM üzerinde nasıl modellendiğini örneklerle gösteriniz. Navigation property / fluent API / annotation kullanım örneklerini ekleyiniz.]

6.5 Migration Yönetimi

[Şema değişikliklerinin migration dosyaları ile nasıl yönetildiğini açıklayınız.]

7. UYGULAMA GELİŞTİRME

7.1 Modüller ve Bileşenler

[Uygulamayı oluşturan ana modülleri ve bileşenleri (örn. Auth Module, Order Module, Product Module) ve aralarındaki bağımlılıkları kısaca açıklayınız.]

7.2 API / Servis Katmanı

[Sunulan REST API uç noktalarını (endpoints) veya servis arayüzlerini özetleyiniz. Örnek istek/yanıt JSON'ları ekleyebilirsiniz.]

Method	Endpoint	Yetki	Açıklama
POST	/api/auth/register	Anonim	Yeni kullanıcı kaydı oluşturur.
GET	/api/products	Tüm Kullanıcılar	Ürün listesini sayfalı şekilde döner.
[...]	[...]	[...]	[...]

7.3 Önemli Kod Parçaları

[Projenin temel iş mantığını gösteren 3-4 önemli kod parçasını (servis sınıfı, repository metodu, controller action vb.) ekleyiniz. Kod parçalarının altında kısa açıklama bulunsun.]

8. BİRİM TEST (UNIT TEST) ÇALIŞMALARI

Proje, geliştirme süreci boyunca birim testler ile desteklenmelidir.

8.1 Test Stratejisi

[Hangi test çatısının (xUnit, NUnit, JUnit, pytest, Jest vb.) kullanıldığını açıklayınız.]

8.2 Test Senaryoları

[Yazılan unit testlerin listesini aşağıdaki tabloya ekleyiniz.]

ID	Test Edilen Sınıf / Metot	Senaryo	Beklenen Sonuç
UT-01	UserService.Register	Geçerli e-posta ve 8+ karakter şifre ile kayıt.	Kullanıcı oluşturulur.
UT-02	UserService.Register	Var olan e-posta ile kayıt denemesi.	Exception fırlatılır.
UT-03	[...]	[...]	[...]

9. BONUS ÖZELLİKLER

Bu bölümde, ders kapsamında bonus puan kazandıran özelliklerden hangileri uygulanmışsa sadece o kısımlara yer verilecektir.

10. TAKIM ÇALIŞMASI VE GÖREV DAĞILIMI

10.1 Takım Üyeleri ve Roller

[Aşağıdaki tabloda takım üyelerinin proje süresince üstlendiği temel roller belirtilecektir.]

Takım Üyesi	Öğrenci No	Temel Rol
[Ad Soyad 1]	[Öğrenci No]	[Proje Lideri / Scrum-Kanban Master / Backend Geliştirici]
[Ad Soyad 2]	[Öğrenci No]	[Backend / Veritabanı – ORM Sorumlusu]
[Ad Soyad 3]	[Öğrenci No]	[Frontend Geliştirici / Test Sorumlusu]

11. SONUÇ VE DEĞERLENDİRME

11.1 Elde Edilen Kazanımlar

[Proje sürecinde edinilen teknik ve takım çalışması kazanımlarını özetleyiniz (ORM/Repository Pattern, Kanban, Git/GitHub akışı, code review kültürü, unit test yazımı, mimari ve tasarım desenleri vb.).]

11.2 Karşılaşılan Zorluklar ve Çözümleri

[Süreç boyunca karşılaşılan teknik veya organizasyonel zorlukları ve uyguladığınız çözümleri belirtiniz.]

11.3 Gelecek Çalışmalar

[Projenin sonraki adımlarda nasıl genişletilebileceğine dair önerilerinizi yazınız (yeni özellikler, performans iyileştirmeleri, mobil sürüm, ek AI senaryoları vb.).]

12. KAYNAKÇA VE EKLER

12.1 Kaynakça

[Projede yararlanılan kitap, makale, çevrim içi kaynak ve dokümantasyonları aşağıdaki gibi listeleyiniz.]

- [Yazar(lar). (Yıl). Eser adı. Yayınevi/URL.]
- [...]

12.2 Ekler

[Tüm proje kodu metin halinde uygun başlıklarla ve arayüz ekran görüntüleriyle eklenecektir. Resim olarak sadece arayüz ekran görüntüleri eklenecektir, kod kısmı metin halinde eklenecektir.]

- EK-A: [Ek başlığı]
- EK-B: [Ek başlığı]